

# PostgreSQL Architecture & Setup

Week 2 – CST4714 Database Administration  
(Supabase focus)

September 4, 2025



*Open  source database power meets modern development*

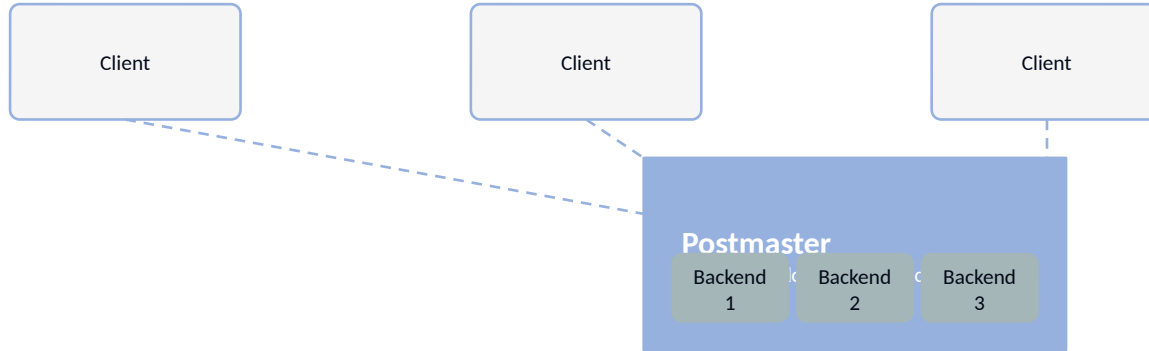


# Agenda & Learning Objectives

**By the end of this lecture you will be able to:**

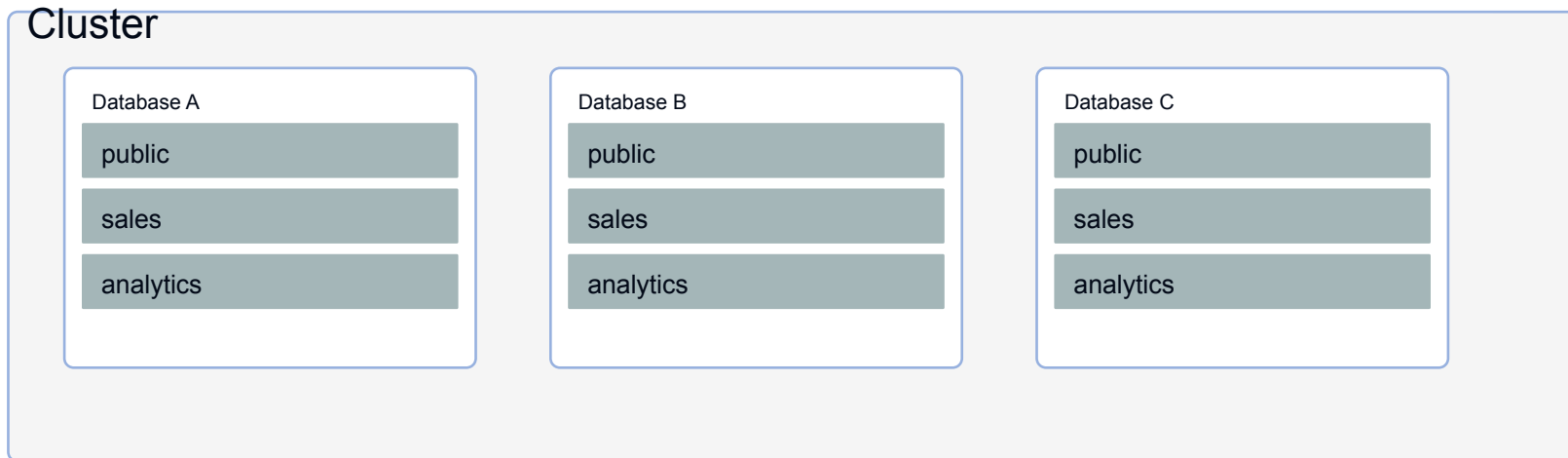
- Understand the PostgreSQL client-server architecture
- Describe clusters, databases & schemas
- Illustrate the physical structure & shared memory
- Identify key background processes
- Explain Write-Ahead Logging (WAL)
- Review transactions & ACID properties
- Visualise MVCC & concurrency control
- Explore system catalogs (data dictionary)
- Discover Supabase architecture & components
- Demonstrate Supabase CLI usage

# Client–Server Architecture



- PostgreSQL runs as a dedicated server process (postmaster)
- The postmaster initialises shared memory, listens for connections and supervises child processes
- Each client connection is handled by its own backend process; processes are isolated and cannot interfere with each other

# Cluster, Database & Schema



- A cluster is a directory tree containing data files, WAL logs and subdirectories for each database
- Each database is isolated within the cluster and holds its own set of schemas
- Schemas organise tables, views and functions within a database

# Physical Structure on Disk

## Data directory (cluster)

**base/**

Databases with table & index files

**pg\_wal/**

Write-Ahead Log segment files

**pg\_global/**

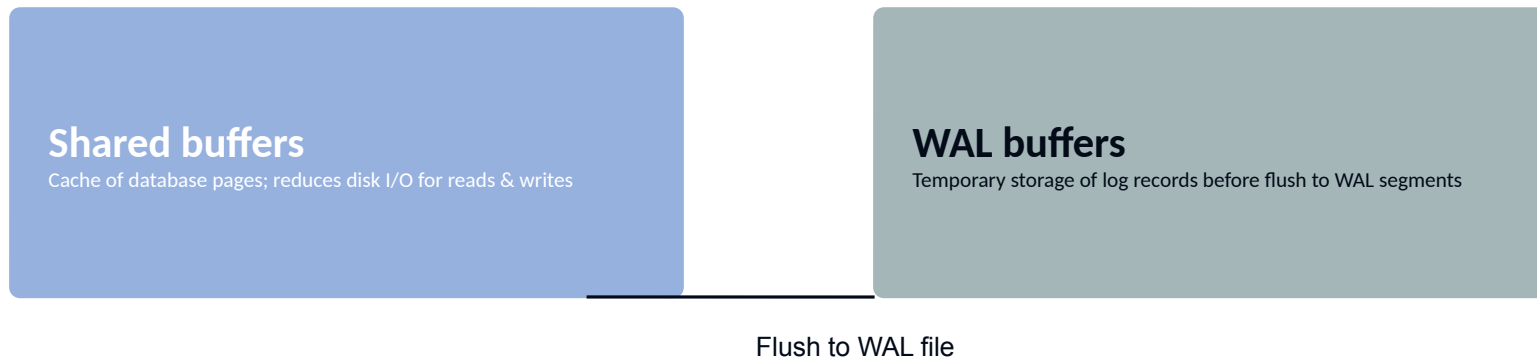
Cluster-wide tables

**pg\_stat**

Statistics & temporary files

A PostgreSQL cluster is a directory tree on disk. Each database lives under the `base/` subdirectory as its own folder. WAL segment files reside in `pg_wal/` and track every change for durability. Global tables and shared metadata live in `pg_global/`.

# Shared Memory & Buffers



- Shared buffers speed up reads by storing frequently accessed pages
- WAL buffers accumulate log records before flushing them to disk
- Both live in PostgreSQL's shared memory, configured at startup

# Background Processes (Part 1)

## **Autovacuum**

Reclaims space from dead rows and updates planner statistics to prevent table bloat

## **Background writer**

Periodically flushes dirty buffers to disk to smooth out checkpoint I/O

## **WAL writer**

Ensures WAL buffers are flushed frequently to maintain durability without writing every data page

## **Checkpoint**

Flashes dirty pages & WAL to disk at checkpoints to bound recovery time

# Background Processes (Part 2)

## **Stats collector**

Gathers activity metrics for the planner and monitoring tools

## **Logger**

Records system messages and errors for diagnostics

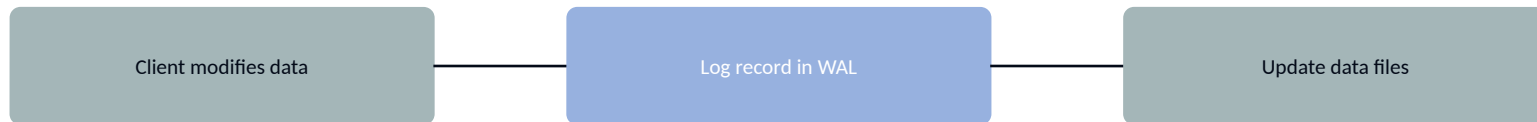
## **Archiver**

Moves completed WAL segments to long-term storage for PITR & replication

## **WAL receiver & sender**

Transfer WAL records between primary and standby servers

# Write-Ahead Logging (WAL)



- All changes are first written sequentially to the WAL, then applied to data files
- Sequential logging reduces random disk I/O and improves crash recovery
- On a crash, WAL is replayed to reconstruct consistent data pages

# Transactions & ACID

## Atomicity

All operations in a transaction succeed or fail together

## Consistency

Transactions bring the database from one valid state to another

## Isolation

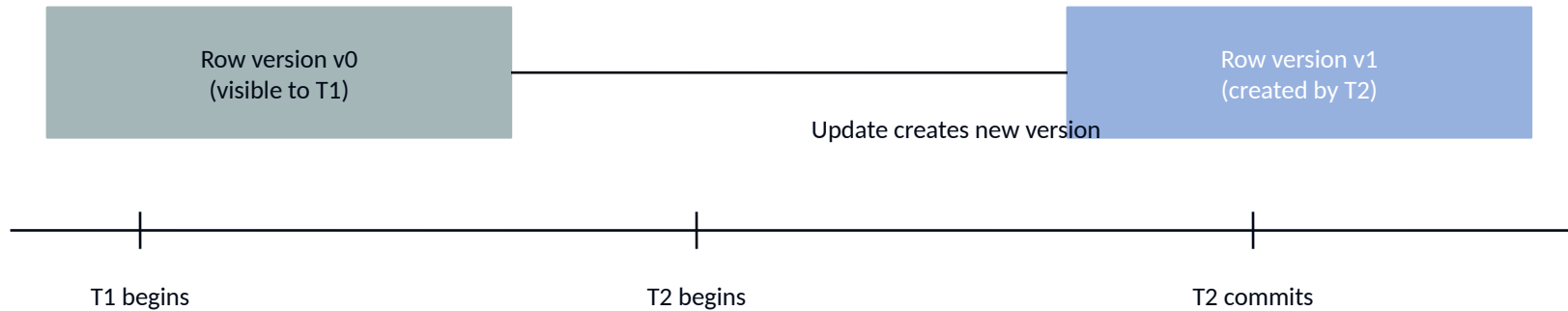
Concurrent transactions cannot see intermediate states

## Durability

Once committed, data changes survive crashes

```
BEGIN;  
UPDATE accounts SET balance  
= balance - 100 WHERE id=1;  
UPDATE accounts SET balance  
= balance + 100 WHERE id=2;  
COMMIT  
;
```

# Multi-☐ Version Concurrency Control



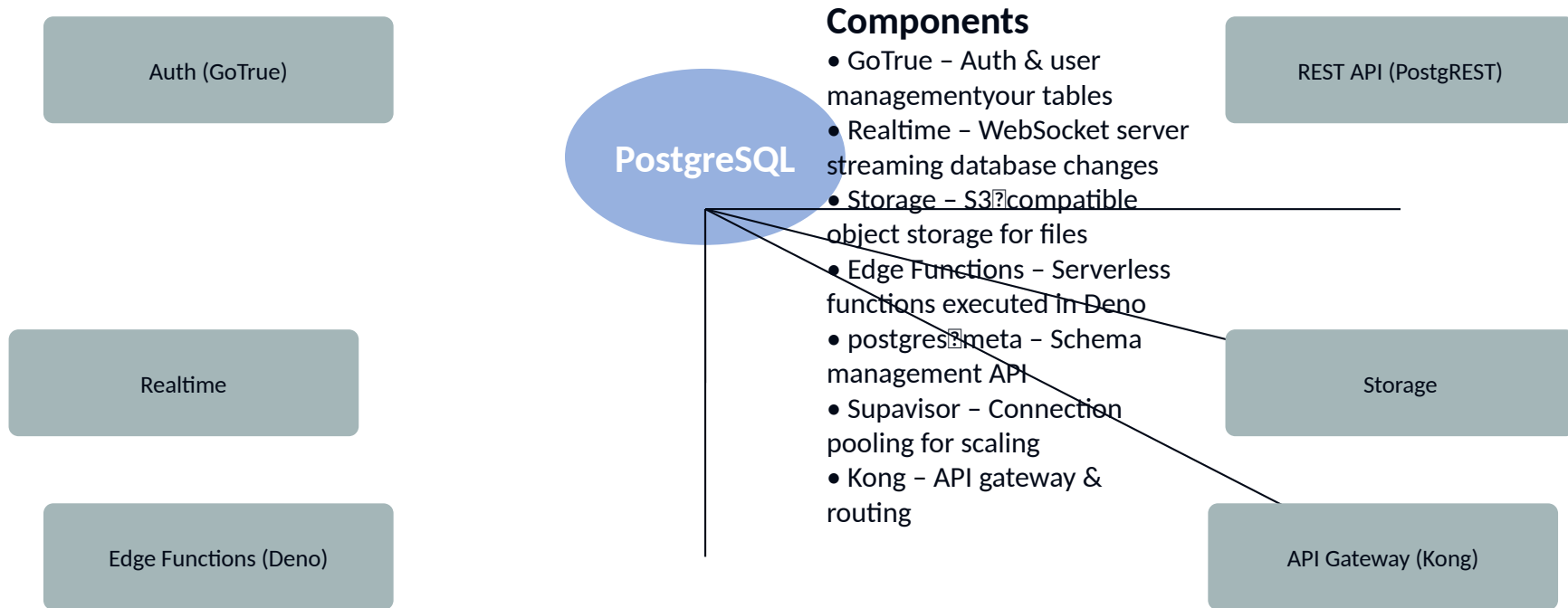
- MVCC stores multiple versions of a row and uses snapshots to provide consistent reads
- Writers create new versions rather than overwriting; readers continue seeing the old version until they commit
- VACUUM eventually cleans up dead row versions

# System Catalogs & Data Dictionary

| Catalog             | Description                                  |
|---------------------|----------------------------------------------|
| <i>pg_class</i>     | One row per table, index or sequence         |
| <i>pg_attribute</i> | Column definitions (name, type, nullability) |
| <i>pg_type</i>      | Data type definitions                        |
| <i>pg_namespace</i> | Schemas within databases                     |

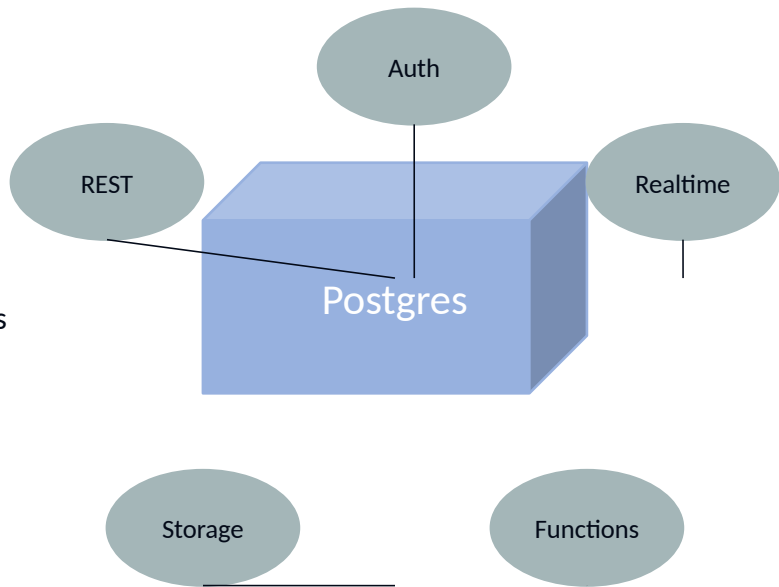
- System catalogs are ordinary tables storing metadata about every object in the database
- They include information about tables, columns, types, indexes and more
- Do not modify catalog tables directly; use DDL commands instead

# Supabase Architecture Overview



# Supabase & PostgreSQL

- Supabase uses PostgreSQL as its core database – your SQL skills apply directly
- PostgREST exposes your tables as REST endpoints without writing controllers
- GoTrue provides JWT-based authentication; integrates with row-level security
- Realtime streams database changes via WebSockets using logical replication
- Supabase Studio and CLI simplify management, but you can always connect with psql



# Supabase CLI Demo

```
# Install the Supabase CLI
npm install -g supabase

# Initialise a new project
supabase init

# Start the local Supabase stack (PostgreSQL, auth, API etc.)
supabase start

# Connect to the local Postgres database using psql
supabase db connect
```

## Install

Get the CLI via npm – it only takes a moment

## Initialise

Create a new project with configuration files and a local database

## Start

Launch Postgres, GoTrue, PostgREST and Realtime locally so you can develop offline

## Connect

Open a psql shell to the running Postgres instance and execute SQL

# Summary & Key Takeaways

- PostgreSQL uses a client-server architecture with isolated backends
- Clusters contain multiple databases; each database groups schemas & objects
- Physical files, shared buffers and WAL segments underpin performance & durability
- Autovacuum, writers and checkpointers work continuously behind the scenes
- Transactions obey ACID; MVCC enables high concurrency
- System catalogs provide metadata; never modify them directly
- Supabase builds on Postgres, adding auth, REST, realtime & storage services

# Discussion & Questions

1. What tasks does the autovacuum process perform and why is it important?
2. How does MVCC allow concurrent reads and writes without locking?
3. Explain the difference between a cluster, a database and a schema.
4. During a crash recovery, what role does WAL play?
5. How does Supabase expose PostgreSQL to the developer while adding value?

# Resources & Next Steps

- PostgreSQL Wikibook: Architecture, Processes, RAM & Files; Transactions & MVCC
- PostgreSQL Manual: Chapter 'Overview of PostgreSQL Architecture'
- Explore the system catalogs using psql: try "SELECT \* FROM pg\_class LIMIT 5;"
- Read Supabase docs on architecture, authentication, storage and realtime
- Experiment with the Supabase CLI on your machine

# Relational Model & Keys



- A relational database stores data in tables (relations) with rows and columns
- Each table has a primary key (PK) that uniquely identifies each row
- Foreign keys (FK) enforce relationships between tables, referencing a primary key in another table
- Relationships support JOIN operations and maintain referential integrity

# Normalization & Normal Forms

| Normal Form | Definition                                                                                     |
|-------------|------------------------------------------------------------------------------------------------|
| 1NF         | All attributes contain only atomic (indivisible) values; no repeating groups                   |
| 2NF         | Table is in 1NF and every non-key attribute is fully dependent on the entire primary key       |
| 3NF         | Table is in 2NF and all attributes depend only on the primary key (no transitive dependencies) |

Normalization is a design process that minimises redundancy and prevents update anomalies  
Ensuring 1NF eliminates repeating groups by making attributes atomic  
2NF removes partial dependencies on a composite primary key  
3NF eliminates transitive dependencies so that non-key attributes depend only on the key

# Data Types & Constraints

| Category  | Examples                          |
|-----------|-----------------------------------|
| Numeric   | integer, bigint, numeric, decimal |
| Character | char(n), varchar(n), text         |
| Boolean   | boolean                           |
| Date/Time | date, timestamp, interval         |

## Constraints control acceptable values in a table:

- NOT NULL – column must contain a value
- UNIQUE – values in a column or group of columns must be distinct
- CHECK – values must satisfy a Boolean expression (e.g. price > 0)
- FOREIGN KEY – ensures references to existing rows in another table

# Index Types & Use Cases

| Index Type | Use Cases                                                                       |
|------------|---------------------------------------------------------------------------------|
| B-tree     | Default index; supports equality and range queries; can return sorted results   |
| Hash       | Fast equality comparisons; rarely used because B-tree covers most cases         |
| GiST       | Generalised search tree; supports geometric data and full-text search           |
| GIN        | Inverted index; excellent for array columns and full-text search                |
| BRIN       | Block range index; summarises large tables; efficient on naturally ordered data |

Choosing the right index improves query performance by orders of magnitude

B-tree is the general-purpose index used for most workloads

Specialised indexes like GiST and GIN accelerate spatial and text searches

# Query Planning & EXPLAIN

PostgreSQL devises a plan for each query before execution.

Plans are composed of plan nodes such as sequential scans, index scans, joins and aggregations.

The optimiser estimates startup and total costs and chooses the most efficient plan.

Use the EXPLAIN command to inspect planned operations and identify bottlenecks.

```
EXPLAIN SELECT * FROM orders  
WHERE customer_id = 42;
```

```
Seq Scan on orders  
(cost=0.00..24.50 rows=5 width=48)  
  Filter:  
    (customer_id = 42)
```

Reading plans is an art: identify full table scans vs index scans, nested loops vs hash joins

Look at cost estimates to understand planner decisions

# Triggers & Replication

Triggers execute a function when events occur (INSERT, UPDATE, DELETE). They can fire BEFORE or AFTER the event and be row-level or statement-level. Use triggers to enforce business rules, maintain audit logs or cascade changes.

Logical replication extends Postgres beyond physical replication. Publications broadcast changes; subscribers receive and apply them via a publish-subscribe model. Logical replication enables online migrations, live version upgrades and real-time analytics.

